4.g Classes - Object Declarations & Expressions

Singletons, Companion Objects & Anonymous Objects in Kotlin

Contents

1 **Object Declarations**
Singletons with the object keyword, data objects for sealed hierarchies

2 **Companion Objects**
Class-level members, factory methods, interface implementation

3 **Object Expressions**
Anonymous objects, inheriting classes, implementing interfaces

4 **Behavior Differences**
Initialization timing, scope, and declarations vs. expressions

Overview: Objects in Kotlin

In Kotlin, the **object** keyword defines a class and creates an instance of it in a single step.

Two key approaches:

Object Declarations

Create named singletons. Always have a name after the object keyword. Thread-safe, lazily initialized.

Object Expressions

Create anonymous, one-time objects. No name needed. Executed and initialized immediately.

Shared Resources

Ensure only one instance exists (e.g., database connection pool, logger, configuration).

Factory Methods

Companion objects define class-level functions to create instances efficiently.

Object Declarations (Singletons)

An object declaration always has a name after the `object` keyword. It defines a class and creates its single instance.

```
object DataProviderManager {
    val allDataProviders = mutableListOf<DataProvider>()
    fun registerDataProvider(provider: DataProvider) {
        allDataProviders.add(provider)
    }
}

val provider1 = DataProvider("Provider1")
val provider2 = DataProvider("Provider2")

DataProviderManager.registerDataProvider(provider1)
DataProviderManager.registerDataProvider(provider2)

println(DataProviderManager.allDataProviders)
//Output: [DataProvider(name='Provider1'), DataProvider(name='Provider2')]
```

Key Rules

- Not an expression
- Cannot be local
- Thread-safe init
- Lazily initialized
- Can extend classes
- Can implement interfaces

Object Declarations: Inheritance

Object declarations can extend classes and implement interfaces, making them singleton instances of their parent types.

```
object Counter {
    private var count: Int = 0
    fun currentCount() = count
    fun increment() { ++count }
}

println("Counter: ${Counter.currentCount()}")
Counter.increment()
println("Counter: ${Counter.currentCount()}")
```

```
Output:
Counter: 0
Counter: 1
```

```
object ReverseComparator : Comparator<String> {

    override fun compare(
        o1: String, o2: String
    ) = o1.reversed().compareTo(o2.reversed())
}

fun testObjectDecl2() {
    val list = listOf("Armando", "Ana", "Alberto", "Alfredo")
    println(list.sortedWith(ReverseComparator))
```

```
Output:
[Ana, Alfredo, Armando, Alberto]
```

Important Restrictions:

- Object declarations **cannot be local** (not nested directly inside a function)
- They **are not expressions** — cannot be used on the right-hand side of an assignment
- Can be nested into other object declarations or non-inner classes

Data Objects

Marking an object with **data** gives it a clean toString() returning its name instead of a hash code.

```
// Plain object
object MyObject

println(MyObject)
// Output: ex4_classes.MyObject@4a574795
```

```
// Data object
data object MyDataObject

println(MyDataObject)
// Output: MyDataObject
```

Compiler generates for data objects:

`toString()`	`equals()`	`hashCode()`
Returns the object name	All instances are equal	Consistent hash

Note: You cannot provide a custom equals or hashCode implementation for a data object.

Data Objects in Sealed Hierarchies

Data objects maintain symmetry with data classes inside sealed hierarchies, providing clean toString().

```
sealed interface ReadResult

data object EOF : ReadResult

data class Number(
    val number: Int
) : ReadResult

data class Text(
    val value: String
) : ReadResult


val result1: ReadResult = Number(42)
val result2: ReadResult = Text("Hello")
val result3: ReadResult = EOF
println(result1) // Output: Number(number=42)
println(result2) // Output: Text(value=Hello)
println(result3) // Output: EOF
```

Why data object?

- Consistent toString() with data class siblings
- Clean logging output
- Useful in sealed hierarchies for state representation

```
Output:
Number(number=42)
Text(value=Hello)
EOF
```

Companion Objects

An object declaration inside a class marked with **`companion`**. Members are called using the class name as qualifier.

```
class MyClass private constructor(private val x: Int) {
    companion object Factory {
        fun create(n: Int): MyClass = MyClass(n)
    }

    override fun toString(): String {
        return "x = $x"
    }
}

val instance1 = MyClass.create(10)
println(instance1)
val instance2 = MyClass.create(20)
println(instance2)

val instance3 = MyClass(1)
// Kotlin error: Cannot access 'constructor(x: Int): MyClass': it is
private in 'ex4_classes.MyClass'.
```

Key Facts

- Name is optional (defaults to **`Companion`**)
- Only one per class
- Initialized when the class is loaded
- Looks like static but are instance members
- Can inherit classes and implement interfaces

Note: There is no **`static`** keyword in Kotlin. Companion objects replace Java's static members.

Companion Objects: Interfaces

Companion objects can implement interfaces — enabling powerful patterns like factory methods with polymorphism.

```
interface Factory<T> {
    fun create(name: String): T
}


class User2(val name: String) {
    companion object : Factory<User2> {
        override fun create( name: String ): User2 = User2(name)
    }

    override fun toString(): String {
        return "User2(name='$name')"
    }
}

// Use as Factory:
val f: Factory<User2> = User2
println(f.create("Alice")) // Output: User2(name='Alice')
```

Java Interop

Use `@JvmStatic` to generate real static methods/fields on the JVM.

Without it, Java callers must use:

`MyClass.Companion.create()`

With @JvmStatic, Java calls:

`MyClass.create()`

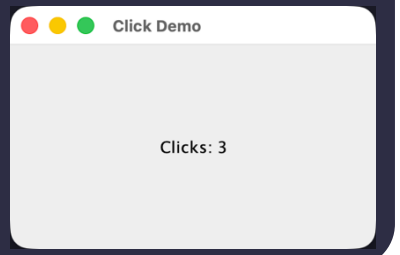
Object Expressions

Object expressions declare a class and create an instance without naming either. They start with the **object** keyword.

```
// From scratch
val obj = object {
    val x = 10
    val y = 20
}
println(obj.x + obj.y)  // Output: 30
```

```
// swing application
fun main() = SwingUtilities.invokeLater {
    var clickCount = 0
    val statusLabel = JLabel("Clicks: 0", SwingConstants.CENTER)

    JFrame("Click Demo").apply {
        defaultCloseOperation = JFrame.EXIT_ON_CLOSE
        setSize(300, 200)
        setLocationRelativeTo(null)
        add(statusLabel)
        addMouseListener(object : MouseAdapter() {
            override fun mouseClicked(e: MouseEvent) {
                statusLabel.text = "Clicks: ${++clickCount}"
                println(statusLabel.text)
            }
        })
        isVisible = true
    }
}
```



Object expressions can:

- Be created from scratch (anonymous object with properties and methods)
- Inherit from existing classes (pass constructor parameters if needed)
- Implement interfaces (even multiple interfaces simultaneously)
- Are **not singletons** — a new instance is created each time

Object Expressions: Scope & Visibility

Object expressions can access variables from the enclosing scope. Unlike Java, this is not restricted to final variables.

```
fun countClicks(window: JComponent) {
    var clickCount = 0

    window.addMouseListener( object : MouseAdapter() {
        override fun mouseClicked( e: MouseEvent) {
           clickCount++
        }
    })
}
```

Type Visibility Rules

- **Private** function returning anonymous object: actual type is the anonymous type — members are accessible
- **Public** function returning anonymous object: type becomes the declared supertype, or Any — added members are NOT accessible

Key difference from Java:

Kotlin can modify enclosing var variables (Java requires final).

Behavior Differences

Object declarations, companion objects, and object expressions differ in initialization timing and lifecycle.

Feature	Object Declaration	Companion Object	Object Expression
Initialization	Lazily, at first access	When class is loaded	Immediately, when used
Singleton?	Yes, single instance	Yes, one per class	No, new each time
Named?	Yes, always named	Optional (Companion)	No, anonymous
Best For	Global shared state	Static-like members	One-time handlers

Thread Safety: Object declaration initialization is thread-safe and done at first access.

Summary

Object Declarations

- Named singletons
- Lazy, thread-safe init
- Can extend classes
- Use `data object` for clean toString()
- Not local, not expressions

Companion Objects

- Replace Java static
- One per class
- Can implement interfaces
- @JvmStatic for Java interop
- Init when class loads

Object Expressions

- Anonymous objects
- New instance each time
- Access enclosing scope
- Modify mutable variables
- Immediate initialization